Rami Lameche

Rotational Mount Documentation

Locate MATLAB class ThorlabRotationalMount.m and motor driver template function called MotorTemplate.m

Note: MotorTemplate.m function is essentially "handle" to the rotational mount, so if you need to add another rotation mount, just create another function using MotorTemplate.m with whatever name you desire. change the variable SN to whatever serial number that corresponds to your motor driver.

Code for MotorTemplate.m:

```
1.      function handle=MotorTemplate()
2.      %methods and functions
3.      %% Create Matlab Figure Container
4.      fpos    = get(0,'DefaultFigurePosition'); % figure default position
5.      fpos(3) = 600;% figure window size;Width
6.      fpos(4) = 400; % Height
7.      fpos(1) = 1000; % Left
8.      fpos(2) = 40; % bottom
9.      f = figure('Position', fpos,...
10.     'Menu','None',...
11.     'Name','Optical Head');
12.     %% Create ActiveX Controller
13.     handle = actxcontrol('MGMOTOR.MGMotorCtrl.1',[0 0 600 400 ], f);
14.     %% Initialize
15.     % Start Control
16.     handle.StartCtrl;
17.
18.     % Set the Serial Number
19.     SN = 12345789; % put in the serial number of the hardware
20.     set(handle,'HWSerialNum', SN);
21.
22. % Indentify the device
23. handle.Identify;
24.
25. pause(5); % waiting for the GUI to load up;
26. end 27.
```

Actxcontrol() function returns handle to the motor using program id 'MGMOTOR.MGMotorCtrl.1' MATLAB won't be able to detect the program id without Thorlabs APT driver.

Download and install Thorlabs APT drivers: (https://www.thorlabs.com/software_pages/ViewSoftwarePage.cfm?Code=Motion_Control&viewtab=1)

To check if it worked use the function actxcontrollist() which shows you a list of Active X program ids and it should say 'MGMOTOR_MGMotorCtrl.1' in the list.

Now Under ThorlabRotationalMount.m change the path variable to directory in which you stored motor functions and class.

```
1.      properties (Access = private) %Only accessible to the class methods
2.      InstrObj
3.      InstruID = '';
4.      Angles = [0,0,0,0,0,0]
5.      Path = 'C:\Users\file';
6.      end
```

Also download SavedAngles.txt and put it the same path directory as the previously stored scripts.

For every MotorTemplate.m created, you must store it in the Obj.InstrObj array as presented in this coding example.

Note: Depending on how you order the Motor functions in the array, the corresponding index is how you access each motor when you use built-in functions.

```
1.      function connect(obj)
2.      obj.Load_Data();
3.      fprintf('Open optical head QWP and HWP GUI and handle...\n');
4.      % Add function for each motor template in the obj.InstrObj list
5.      obj.InstrObj = [MotorTemplate1(),MotorTemplate2()];
6.      fprintf('Moving Home Position as Hardware Initialization...\n');
7.      obj.InstrObj(1).MoveHome(0,true);
8.      obj.InstrObj(2).MoveHome(0,true);
9.      fprintf('Finished...\n');
10.     end
```

Now create .m file to use the ThorlabRotationalMount.m class to create handle.

```
1. clear
2. clc
3.
4. handle = ClassCrossPolPair()
5.
6. handle.connect()
```

Once you execute it you should get popup that looks like this:

If you did everything right, it should automatically connect to the assigned serial number. Depending on how many motors you connected, you should get a popup for each one for them.

You can manually control the rotational mounts this way and you can adjust the stepsize/velocity under settings.

Incase to you wanted to make a custom script instead of using the UI, you can use these useful functions.

get_Angle(i)

Retrieves the current angle of the motor specified by index i.

Rotate_POS(angle, i)

Rotates the motor (specified by index i) to the given absolute angle.

Rotate_Relative(relative_angle, i)

Rotates the motor (specified by index i) by a relative angle, calculated as the current angle plus relative_angle.

Save_Data()

Saves the current angles of all motors, allowing you to return to these positions later.

Go_to_Last_All()

Returns all motors to its previously saved angle in local disk.

Load_Data(i)

Returns individual motor to its previously saved angle given specified index.

CloseConnectionClear()

Closes all handles of each of the motors.

Email: rlameche@terpmail.umd.edu